QCon
全球软件开发大会

InfoQ 技术大会

从生码到可信交付

Harness For Hybrid Coding 实践

何伟 | 阿里巴巴技术专家

一人三端交付的工程实践



QCon
全球软件开发大会

InfoQ 技术大会

目录

01 背景挑战：一人三端交付的复杂性

02 工程方法：统一架构与 Harness 双循环

03 落地实践：一个真实需求的交付过程

04 质量验证：分阶段验证与交付效果

背景：AE 三端业务与 AI 生码提速

三端业务目标相近，AI 编码提速，为一人主导三端交付提供基础

AE 业务背景

AE · 全球速卖通

AliExpress

面向全球消费者的电商平台

App 核心业务场景

电商首页	推荐	搜索
商品详情	购物车	下单
订单	类目	频道

覆盖导购、交易与订单服务等核心场景

三端交付现状

AE App 业务覆盖三端

Android	基于 Android 技术栈 独立实现与迭代
iOS	基于 iOS 技术栈 独立实现与迭代
Msite	H5 技术栈 独立实现与迭代

业务目标相近

框架、代码仓库与发布流程各自独立

AI Vibe Coding

AI 提速，拓展实现能力

编码能力提升

用自然语言描述需求

快速生成、修改与调试代码

多端实现成本下降

降低多端重复编码的时间成本

扩大单个研发可承担的实现范围

交付目标

一人主导一个需求的三端交付

需求澄清、三端实现、验证、集成与发布

背景：一人三端交付的四项挑战

01 需求容易遗漏

业务规则、异常与边界未说全
需求传给各端时，信息还可能丢失

02 三端实现容易不一致

同一需求在三个仓库分别实现
规则理解、职责划分与交互可能不同

03 单人三端协作，注意力分散

逐端对话式生码，需分别实现与验证
反复切换上下文，协调与检查占用精力

04 验收标准难统一，验证复杂度高

三端需要共同标准，也要保留端差异
既要验本次改动，也要验真实链路与存量影响

生码变快后，如何兼顾三端交付效率与全生命周期质量？

QCon
全球软件开发大会

InfoQ 技术大会

目录

01 背景挑战：一人三端交付的复杂性

02 工程方法：统一架构与 Harness 双循环

03 落地实践：一个真实需求的交付过程

04 质量验证：分阶段验证与交付效果

整体方案：统一架构与 Hybrid Coding Agent

统一架构

六层职责、依赖约束与验证边界
形成三端共同遵循的架构规范

Hybrid Coding Agent

双循环 Harness：外环编排、内环执行
按统一规范完成重构、生成与自验证

达成路径 双循环 Harness 按统一架构与流程规范完成三端重构

交付结果 形成 Agent 主导的自动化交付流程

01 需求阶段

明确业务规则、数据契约
提前约定验收标准

→

02 生成阶段

按统一职责与规格实现
自验证并逐端核对

→

03 验证阶段

研发：功能与细节正确
集成：接口与链路完整
发布：回归确认稳定

→

04 知识更新

将确认的业务规则、修复经验与验证用例更新到知识与工程规范，供后续需求复用

共同规格保障质量，统一编排减少人工切换与逐端跟进

三端六层架构：职责、实现与验证

同一份需求，三端按共同职责分别实现，以共同契约和验收条件验证

共同规范 Spec： 业务规则 · 请求/响应契约 · 状态与错误语义 · 端差异 · 验收条件

层级职责	Android	iOS	Msite / H5	针对性验证
UI 展示状态，派发事件	Activity / Fragment / ViewHolder 订阅 StateFlow / LiveData 点击通过回调交给 ViewModel	ViewController / View / Cell 渲染状态，delegate / block 上报 页面业务入口收敛到 PageViewModel	React 页面 / 组件 / 楼层 Selector / useAppData 读取状态 dispatch / useAppAction 派发事件	**运行时评测** 布局、显隐、操作与事件绑定 加载 / 空态 / 错误态 / 重试
ViewModel 组织展示状态，协调调用	ViewModel + StateFlow / LiveData 维护 loading / data / error 调用 UseCase，不直接访问 Repo	PageViewModel / FloorViewModel 页面协调 / 楼层数据转换 delegate 通知，不持有 UIKit 视图	Redux Model + Store + Hooks 页面：useAppData / useAppAction 楼层：独立 Store + createSlice	**页面状态测试** 事件 → 调用 → 结果 → 状态变化 重复触发、取消与生命周期
UseCase 封装业务规则，明确结果	纯 Kotlin 业务用例 组合 Repo、校验与流程编排 不依赖 Android UI	Foundation 业务用例 组合 Repo、校验与流程编排 不依赖 UIKit	业务函数 / 异步 UseCase 由 thunk 触发，组合数据访问 规则可脱离 React 独立测试	**规则单测** 正常分支 / 边界 / 异常输出 使用 Fake / Mock 替换外部依赖
Repo 访问数据，隔离外部依赖	Repository 接口 + 实现 网络 / 缓存 / 存储适配 统一数据映射与错误转换	Repository 协议 + 实现 网络 / 缓存 / 存储适配 统一数据映射与错误转换	数据访问封装 MTOP runtime / 缓存 / 存储 统一数据映射与错误转换	**适配测试** 字段映射、错误转换、缓存逻辑 超时与异常响应
Types 定义模型与数据契约	纯 Kotlin 数据类 / 枚举 请求参数、结果、页面状态 不依赖 android.*	Foundation 模型 / 枚举 请求参数、结果、页面状态 不依赖 UIKit	TypeScript type / interface GalaxyFloorItem / RenderResult 约束接口和楼层字段	**契约检查** 字段完整性、空值、类型与结果语义 静态类型 + 运行时协议校验
Config 场景参数、策略与开关	配置聚合 / 场景策略 开关、默认值、组件注册表 隔离配置 SDK	配置聚合 / 场景策略 开关、默认值、组件注册表 隔离配置 SDK	配置 / 场景与组件映射 compName → 组件 / Store 工厂 统一开关、默认值与回退	**组合检查** 配置组合、缺失值、默认值与回退 开关前后行为

调用主链： UI → ViewModel → UseCase → Repo；Types / Config 是支撑依赖，底层不得反向依赖上层。

集成验证： 各端接入真实入口与接口，检查跨层协作、协议解析与最终业务结果

H5 按职责映射到六层；具体目录与依赖规则以业务仓为准。页面协调使用 PageViewModel，楼层使用 FloorViewModel；独立 Store 使用 createSlice。

案例：下单校验的职责、依赖与验证

架构示例：用下单校验说明三端如何划分职责、隔离依赖与安排验证

重构前：同一业务规则分散在不同位置

Android
各楼层 ViewModel 内校验

iOS
Engine 集中校验

Msite
React 校验事件链

统一目标 **三端下单前校验由 SubmitOrderUseCase 承担**

三端分别实现；业务规则面向接口；页面状态和交互分别由 ViewModel、UI 承担

生成职责

UseCase：下单前业务判断
ViewModel：编排提交状态
UI：触发提交、呈现结果
三端按相同职责分别实现

契约与依赖

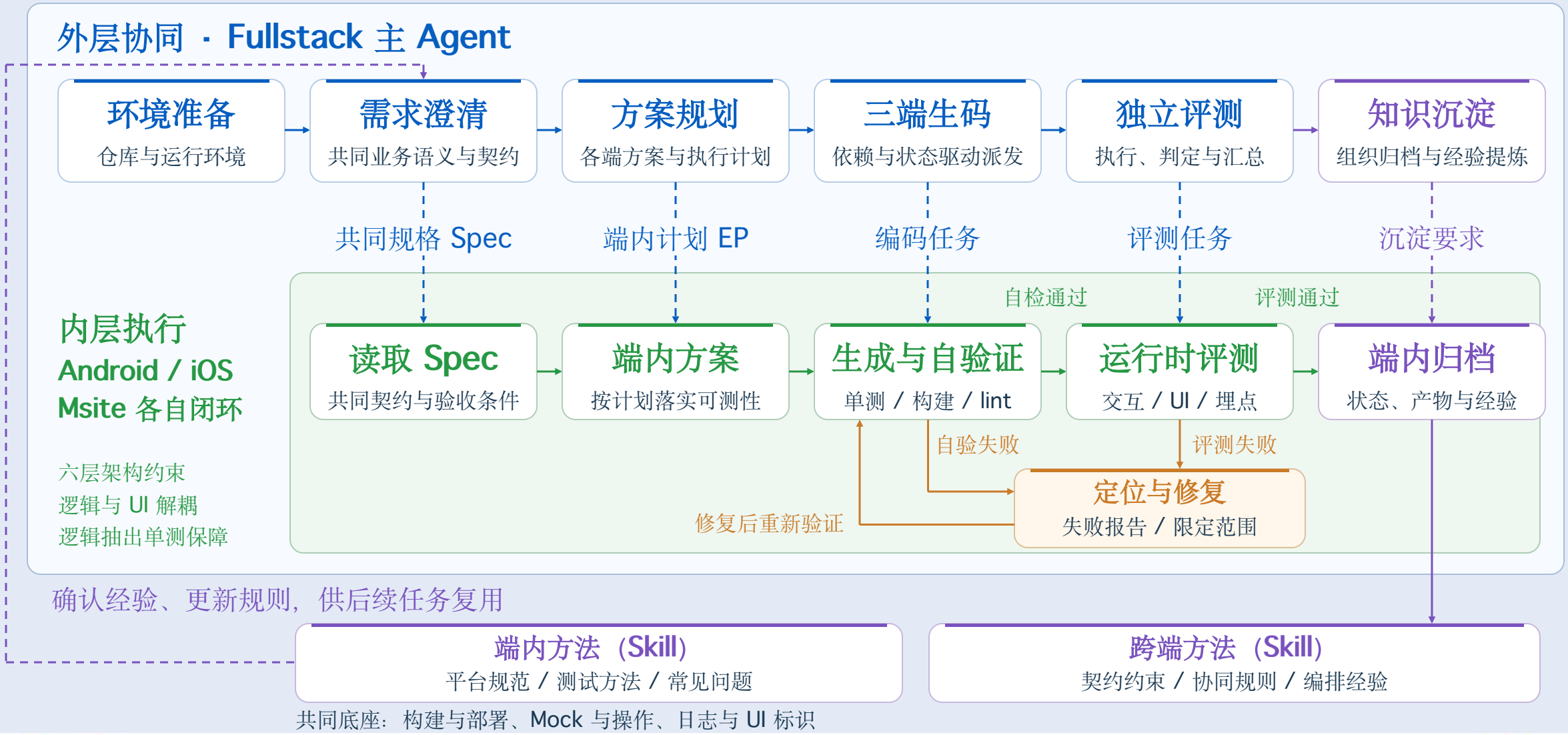
Types：输入与成功 / 失败结果
Config：场景参数与功能开关
需取数时依赖 Repo 接口
单测可用 Fake / Mock 替换 Repo

验证分工

业务分支与异常 → 规则单测
状态变化 → 页面状态测试
契约与配置组合 → 专项检查
提示与用户操作 → 运行时评测

职责与验证边界明确后，再由 Harness 组织三端任务、执行与结果回传

双循环 Harness：外层协同三端，内层生码自验证



Harness 双环工程：主仓编排与子仓执行

主仓管理共同规格与跨端计划，子仓按端内执行计划（EP）完成实现与验证

外环 · Fullstack 主仓

AEFullstack_PlaceOrder/

AGENTS.md	Agent 工作约定
.qoder/	**外环能力**
└ agents/	探索 / 规划 / 生码 / 验收
└ skills/	澄清 / 派发 / 评测 / 归档
docs/	**跨端产物**
└ project.md	三端仓库与能力索引
└ requirements/active/	共同 Spec 与需求契约
└ exec-plans/active/	全局 PLAN、依赖与进度
└ quality/	验收与评测产物
└ solutions/	跨端经验与复用规则
app/	**业务子仓**
└ Android/{repo}/	Android 业务仓库
└ iOS/{repo}/	iOS 业务仓库
└ AHE/{repo}/	跨端页面组件仓库
msite/{repo}/	Msite 业务仓库

内环 · 单端 Repo（Android 示例）

{repo}/

AGENTS.md	端内工作约定与任务路由
.qoder/skills/	**内环能力**
└ ae-android-framework/	平台框架规范
└ ae-harness-plan/	生成端内 EP
└ ae-harness-execute-plan/	生码、单测与自验证
└ ae-harness-eval/	运行评测与交付验收
└ ae-harness-knowledge/	知识更新与持续保鲜
docs/	**端内产物**
└ design-docs/	六层契约与执行约束
└ requirements/	端内需求与验收条件
└ exec-plans/	端内计划、状态与证据
└ product-features/	分层业务知识
└ quality/	质量规则与检查资产
└ learnings/ · references/	经验与协议参考
harness-placeorder/src/	**业务代码与测试**
└ main/ · test/ · androidTest/	实现 / 单测 / 设备测试

下发　需求规格 Spec + 跨端计划 PLAN → 端内需求与 EP

回传　进度、代码产物与验证证据

状态与产物写盘：按 PLAN / EP 恢复未完成任务；{repo} 表示业务仓库

重构流程：从业务梳理到三端实现与上线

以核心业务场景为单位，梳理业务知识，由 Hybrid Harness 组织三端重构与验证

工程前提　六层架构与生成、验证规范　Hybrid Harness 外环协同、内环执行

01	02	03	04	05	06
业务知识梳理	**共同规格确认**	**各端重构规划**	**生成与自验证**	**评测与修正**	**集成与上线**
读取三端实现 分层梳理业务 规则、行为与差异	确认共同要求 保留必要端差异 明确契约与验收	按六层职责拆分 明确复用与改动 约定各端验证	内环按规范实现 执行单测与自验 回传代码与状态	核对功能与契约 按场景执行评测 定向修正与复验	接入真实业务链 完成集成与回归 发布并观察结果
场景知识 行为基线	**共同 Spec** 业务与契约约定	**跨端计划 PLAN** 端内计划 EP	**三端代码** 自验证结果	**评测证据** 修正与复验记录	**上线结果** 可复用经验

下单校验说明职责与验证分工；下面用评价卡片展示交付方法与已有结果

执行中持续反馈与定向修正，上线后将确认的经验回流知识与工程规则，持续保鲜

QCon
全球软件开发大会

InfoQ 技术大会

目录

01 背景挑战：一人三端交付的复杂性

02 工程方法：统一架构与 Harness 双循环

03 **落地实践：一个真实需求的交付过程**

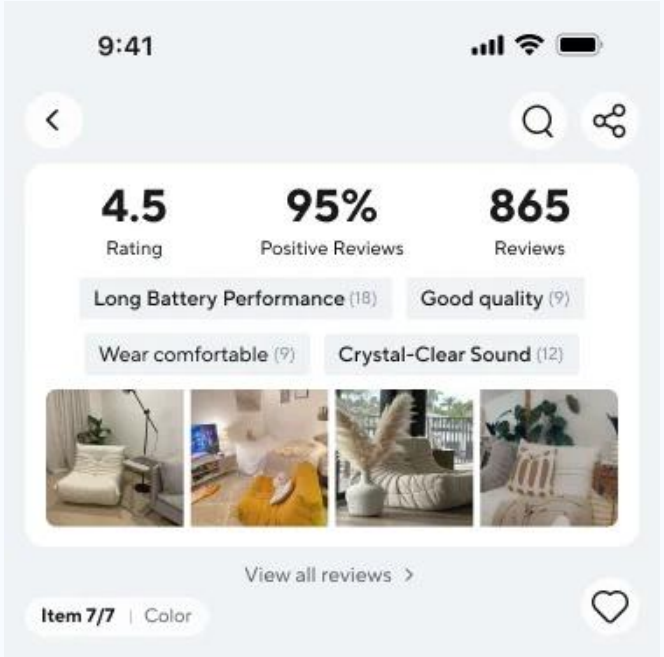
04 质量验证：分阶段验证与交付效果

案例：评价卡片的需求澄清

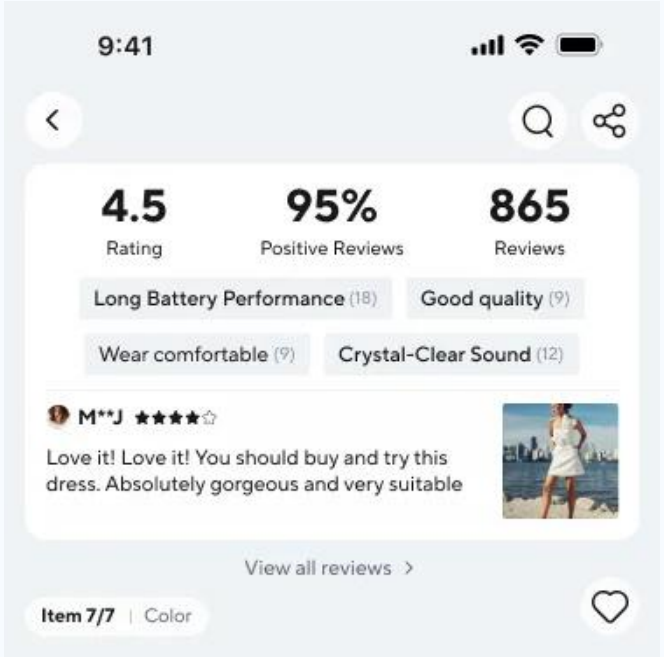
结合设计稿与现有工程，明确三端共同遵守的业务规则

01 需求澄清 —— 02 方案设计 —— 03 方案执行 —— 04 产物核对

大图布局：买家图片 ≥10 张



紧凑布局：买家图片 <10 张



设计输入，非运行截图

澄清并确认验收条件

01 布局条件

按图片数量切换布局
检查 9 / 10 张边界

02 点击行为

使用 action.target
空值不跳转

03 空数据处理

无摘要数据，不建卡片
不注册 tab

参与端：Android、iOS、Msite

本步产物 | 需求说明、设计稿、澄清结论与验收条件

案例：共同规格与三端方案设计

外环维护共同约束，内环设计端内方案，核对通过后进入执行

01 需求澄清 — 02 方案设计 — 03 方案执行 — 04 产物核对

外环主仓

人确认关键决策

位次、布局、跳转与排除项

SPEC-019 共同规格

业务规则、数据契约
验收条件

PLAN-019 整体计划

参与端、任务拆分
产物要求

共同规格与计划进入各端方案设计

内环三端子仓

Android 端内方案

EP-122：模型、策略与解析
明确 UI 接入与验证任务

iOS 端内方案

EP-043：Item VM 与 Cell
明确解析、导航与验证任务

Msite 端内方案

模型、布局与入口接入
明确类型检查与单测任务

方案核对与修正

共同规格 16 个字段 → Android 首轮覆盖 10 个 → EP-122 v2 补齐任务

iOS 对照共同规格补齐字段。需求文件转换遗漏仍待修正。

本步产物 | SPEC-019、PLAN-019、端内方案、契约核对与修订记录

案例：内环按方案执行与自验证

端内 Harness Agent 按方案完成实现与自验证，向外环回传产物

01 需求澄清 —— 02 方案设计 —— 03 方案执行 —— 04 产物核对

读取端内方案 → 分任务修改代码 → 补齐单测与编译检查 → 回传代码与执行记录

Android

- 模型、策略与解析
- UI 与业务接入
- 单测、编译与静态分析
- 回传代码、日志与产物索引

iOS

- Item VM、Cell 与解析
- 页面与导航接入
- 语法检查、单测与模块编译
- 回传代码、日志与产物索引

Msite

- 模型与布局策略
- 页面与入口接入
- 单测、构建与类型检查
- 回传代码、日志与产物索引

外环接收三端产物索引，核对实现、构建与单测结果

本步产物 | 代码改动、单测用例、编译与测试日志、产物索引

案例：外环核对产物与当前结果

按共同规格核对三端实现、构建检查与单测结果

01 需求澄清 —— 02 方案设计 —— 03 方案执行 —— 04 产物核对

	Android	iOS	Msite
核对依据	SPEC-019 的业务规则、数据契约与验收条件		
自验证结果	编译与静态分析通过 新增单测 48 / 48 通过	语法检查 9 / 9 通过 单测、模块编译通过	构建与类型检查通过 布局、边界与跳转单测通过
当前结论	已有实现与自验证证据	已有实现与自验证证据	已有实现与自验证证据

功能验收还需运行证据 核对布局边界、跳转与空数据行为，后续验证章节展开

外环汇总 通过项、待补项与问题，返回对应环节处理

本步产物 | 产物核对记录、当前结论、待补项与问题清单

案例复盘：提炼可复用的任务经验

从 PLAN / EP 的真实记录提炼经验，区分已应用经验与待确认的改进候选

真实反馈	提炼的规则或经验	记录状态
规划时漏派 iOS	逐端核对实际 Agent 调用及回传	已有派发与回传核对记录
共同规格转成端内需求时漏	保留完整契约，与各端计划逐项核对	端内计划已修正 需求文件转换待改进

经确认后，按适用范围治理

端内知识

业务规则、框架用法、修复经验

product-features / learnings

跨端共性

契约传递、协作模式、共性问题

主仓 docs/solutions

Harness 能力

流程规则与 Skill 的改进建议

提出候选，经评审和验证后更新

按工程导航索引检索去重，确认适用条件，更新端内与跨端知识，供后续任务召回

知识与 Skill 更新：回测验证后应用

真实任务暴露问题，经验进入候选规则，经验证后影响下一次生成与执行

任务反馈
问题、证据与根因

→

提炼并更新
候选更新，经审阅

→

固定用例回测
比较结果、过程与成本

→

后续任务复用
加载更新后的约束

后续执行继续产生反馈，代码与工程规则持续保鲜

本次记录中的经验复用

规划漏派 iOS → 记录派发核对经验
已核对实际派发与结果回传
PLAN 保留问题、处置与后续核对记录

回测验证与持续迭代

固定需求与现场，对比规则更新前后版本
比较覆盖与通过、重复问题、耗时与成本
需求转换改进：待补同一用例前后结果

已回传材料还需运行验证；接下来明确证据能支持的结论范围

QCon
全球软件开发大会

InfoQ 技术大会

目录

01 背景挑战：一人三端交付的复杂性

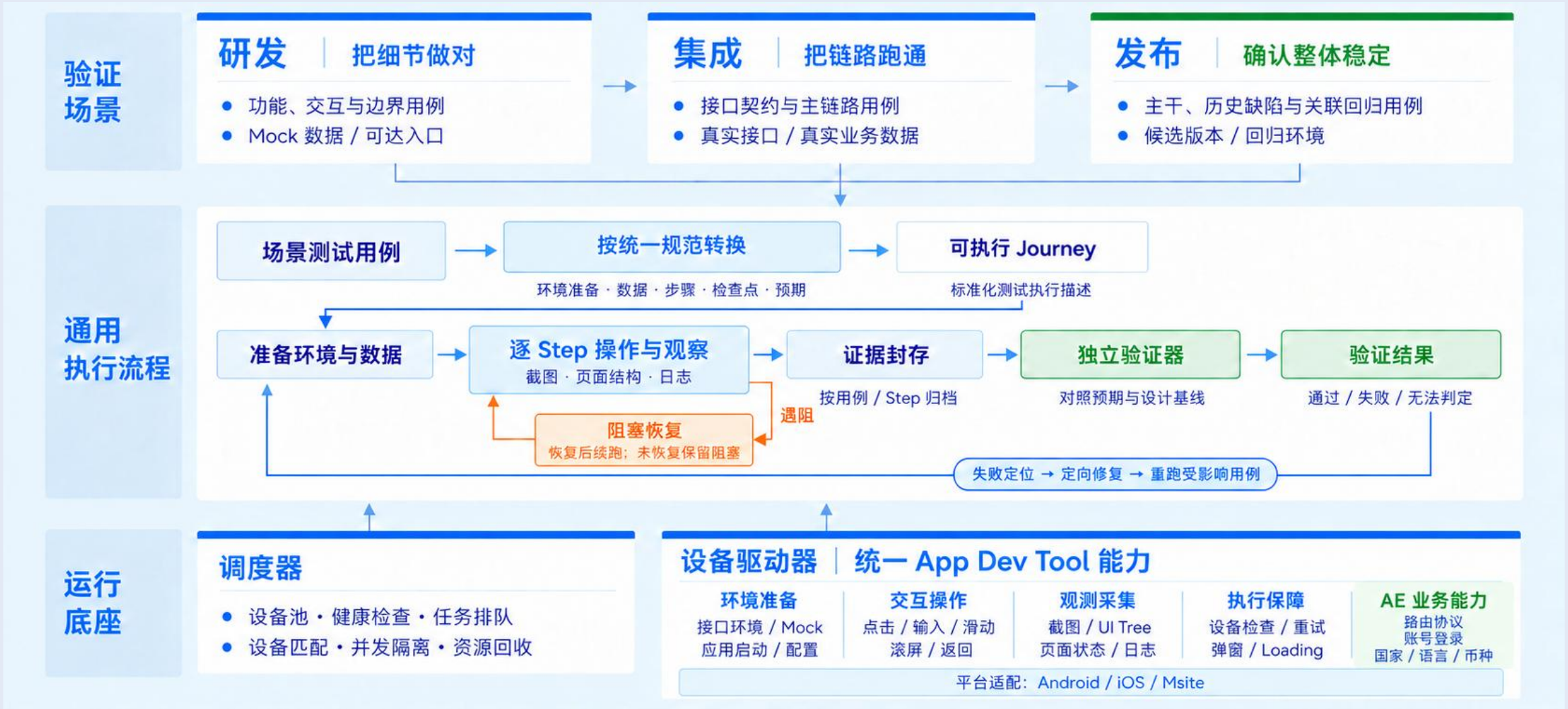
02 工程方法：统一架构与 Harness 双循环

03 落地实践：一个真实需求的交付过程

04 **质量验证：分阶段验证与交付效果**

验证整体方案：三阶段场景与统一执行

图中展开页面与链路场景；逻辑单测独立执行，结果共同汇总。



研发验证：逻辑单测与页面场景

共同输入：Spec、数据契约与验收条件

评价卡片：≥10 张用大图布局，<10 张用紧凑布局；无摘要不建卡片，跳转目标为空不跳转

	检查与准备	执行与具体断言	留存的验证证据
01 逻辑单测 检查规则 与输出状态	构造 9 / 10 张、空数据 及空跳转目标等输入 用 Fake / Mock 隔离依赖	运行单元测试 核对布局选择、卡片创建 与导航触发条件	测试输入与断言结果 失败分支与定位信息 形成单测报告
02 页面场景 检查呈现 与交互绑定	准备 Mock 数据与入口 按用例配置 AE 环境 组织可执行 Journey	操作页面、滑动与点击 核对布局、内容、入口响应 及符合约定的跳转	截图与 UI Tree 操作记录、日志与事件 按用例 / Step 关联保存

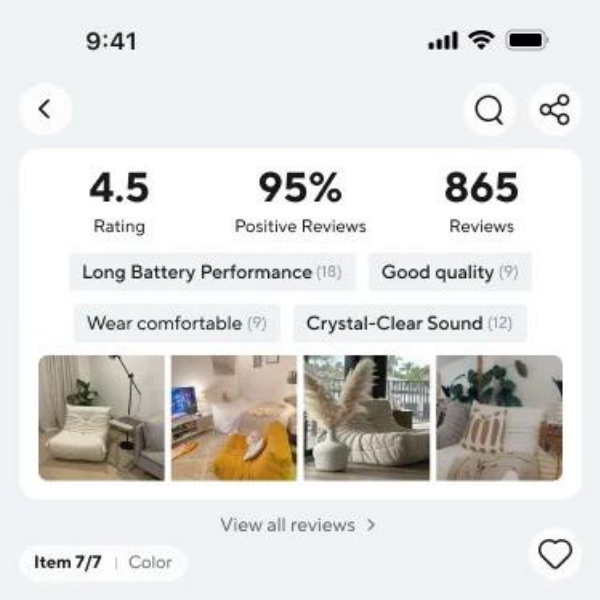
研发结论　两路必测项共同汇总：通过后进入集成验证；失败修复复验；缺证据保留未决

评价卡片：场景用例转换为 Journey

研发验证 / EV-001 大图布局

场景测试用例

买家图片 ≥10 张
展示大图布局、四张图片与评价入口



设计基线

按规范转换为可执行 Journey

环境准备	指定测试版本、Mock 场景与可达入口
测试数据	大图布局数据，图片数量满足 ≥10 张
执行步骤	加载 Mock → 进入详情 → 左滑到评价卡片
检查对象	卡片内容、所在位置、底部独立导航槽位
预期结果	卡片位于主图序列后，底部导航槽位独立

Journey 是标准化执行描述；检查对象继续细化为可判定的断言。

执行与采证：评价卡片案例展开

EV-001 大图布局：从主图左滑到评价卡片，并留下可追溯证据

执行

01 读取 Journey
Step：2-swipe-left

→

02 标准 Dev Tool
驱动设备向左滑动

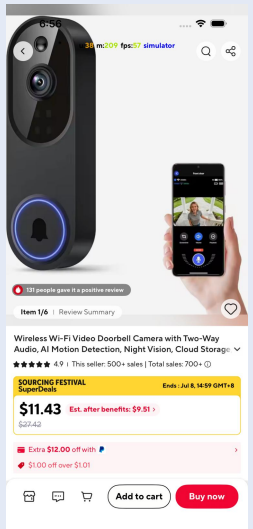
→

03 定位评价卡片
读取状态，完成当前 Step

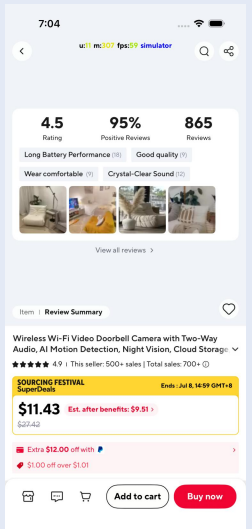
AI 条件介入 定位受阻时观察页面并规划继续左滑，再调用工具执行；未恢复则记录阻塞

截图 动作前后完整页面

操作前



评价卡片可见后



证据目录

按用例 / Step 归档

```
EV-001/
  evidence/2-swipe-left/
    pre-action/
      screenshot-0.png
      ui_tree-0.json
    checkpoint-0/
      screenshot-0.png
      ui_tree-0.json
```

pre-action：动作前
checkpoint-0：动作后

日志 / 执行记录

报告字段摘录

journey-result-EV-001.json

```
case_id: "EV-001"
step_id: "2-swipe-left"
action_status_seen:
  ["COMPLETED"]
screenshot:
  "checkpoint-0/screenshot-0.png"
ui_tree:
  "checkpoint-0/ui_tree-0.json"
sealed: true
```

评价卡片：证据判定与验证结果

按场景检查证据，从 UI、交互、埋点三个维度形成结论

9 / 9
场景通过

12 / 12
UI、交互、埋点

99.86 / 100
综合评分

分类	用例	验证场景	结果
主链路	EV-001	大图布局、卡片位置与导航槽位	通过
主链路	EV-002	紧凑布局、卡片位置与导航槽位	通过
主链路	EV-003	左滑到卡片、点击评价入口导航	通过
负向链路	EV-004	无摘要时隐藏卡片与导航槽位	通过
边界 Case	EV-005	无统计数据时隐藏统计区域	通过
边界 Case	EV-006	无标签时隐藏标签区域	通过
边界 Case	EV-007	无画廊时隐藏图片，采用紧凑布局	通过
边界 Case	EV-008	无评价内容时隐藏评价块与缩略图	通过
负向链路	EV-009	无入口时隐藏入口，其余区域正常	通过

判定依据与评分

UI 99.59

检查：内容、布局与样式

依据：设计基线、截图与 UI 树

交互 100

检查：滑动、点击与导航

依据：操作记录、前后页面与路由

埋点 100

检查：事件、参数与触发次数

依据：埋点规范、关联 Step 日志

(99.59 + 100 + 100) ÷ 3 ≈ 99.86

全部通过　失败 0，未决 0

集成验证方案

以功能链路完整性为主，验证跨页面、跨服务协作

场景覆盖

从研发场景中筛选

核心功能链路与必要分支
明确真实入口、账号、数据与业务结果

跨页面

参数传递、跳转、返回与状态衔接

跨服务

请求上下文、数据契约与服务协作

验证取舍

重点验证

主链路完整跑通
功能协作与最终业务结果

复用研发结果

UI 细节、端内异常与边界组合

按影响补测

可能阻断主链路的关键异常

环境准备

版本与环境

测试包 / H5 集成产物
明确版本、环境与隔离标

账号、数据与设备

准备真实业务条件
明确占用、重置与清理方式

真实接口请求

核验服务配置与实际请求目标

执行与验收

01 **绑定本轮目标**
版本、环境、Case

→

02 **执行与采证**
页面、请求与日志

→

03 **独立判定**
功能链路与业务结果

→

04 **修复与复验**
分流问题，修复后重跑

集成验证案例：评价卡片

沿用已有研发场景，补充真实接口与页面之间的协作检查

本轮真实业务链路

详情入口 → 评价接口 → 卡片呈现 → 评价入口 → 目标评价页

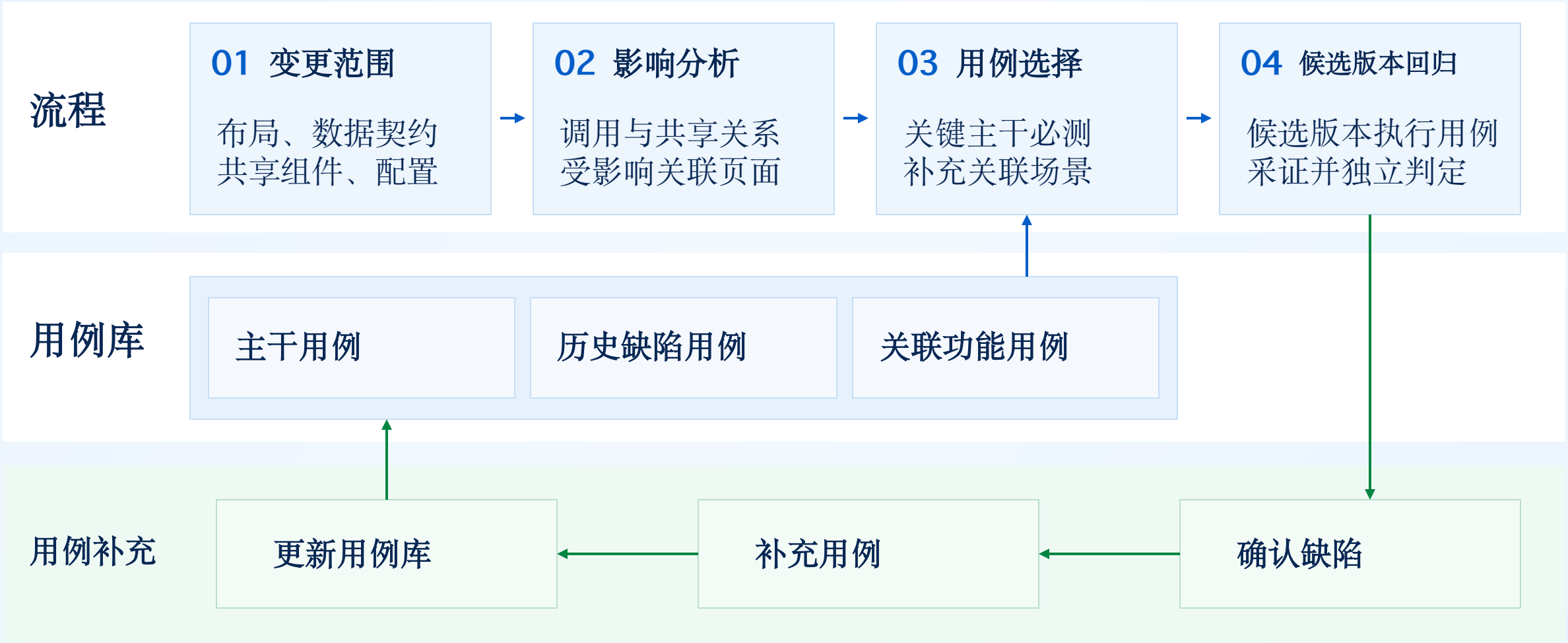
使用指定版本、真实账号与商品数据，核验实际接口请求

已有研发场景	集成阶段增加的检查	对应证据
大图、紧凑布局与卡片位置	真实返回字段的解析、映射与页面呈现	接口响应、页面截图、UI 树
左滑定位、点击评价入口	详情入口商品 ID，跳转参数与目标内容	操作记录、路由记录、目标页面
无摘要及局部内容缺失	真实接口空响应、字段缺失时的页面状态	请求响应、运行日志、页面状态
既有业务步骤与断言	跨页面返回状态、服务超时与错误处理	前后页面、异常日志、业务状态

按变更补充　国家、语言、币种与登录状态差异

执行记录　按 Android、iOS、H5 分别关联版本、环境、Case 和证据

发布验证：回归用例选择


流程
01 变更范围
布局、数据契约
共享组件、配置
02 影响分析
调用与共享关系
受影响关联页面
03 用例选择
关键主干必测
补充关联场景
04 候选版本回归
候选版本执行用例
采证并独立判定
用例库
主干用例
历史缺陷用例
关联功能用例
用例补充
更新用例库
补充用例
确认缺陷

发布验证：回归范围与恢复

基于影响明确性确定回归范围

关键主干	必须覆盖
影响明确	选择关联用例
影响不明确	扩大范围

! 覆盖范围扩大，执行成本增加

选择性回归需记录依据和剩余风险

发布异常处置与恢复验证

异常观测	适用的处置方式	恢复验证
指标、日志、业务检查条件	开关、降级、回滚方案	状态恢复、主链路复验

评价卡片：验证结果与交付依据

三阶段验证与交付依据

研发验证	实现自验证：三端构建检查与单测通过 页面评测：Android / iOS / Msite 均通过
集成验证	已定义真实入口、接口与跳转检查范围 按端记录实际结果与执行证据
发布验证	已说明回归选择与恢复验证方法 记录候选版本、回归范围与恢复验证结果

交付结论汇总三阶段结果，记录验证范围、必测项结果与风险。

问题处置与复验

区分实现 / 环境 / 契约 / 判定问题 → 修复或补证 → 重跑受影响用例 → 更新结论

产品回归验证业务变化；Harness 回测验证规则变化。结果与证据按端、阶段关联归档。

重构成果：三端页面覆盖与实施规模

10 个业务场域完成 Android、iOS、Msite 页面改造与上线

10
完成改造的业务场域

95%
改造页面覆盖的页面流量

65.48 万行
三端累计代码量

平台	重构后代码量	占比
Android	250,335 行	38.2%
iOS	240,092 行	36.7%
Msite	164,336 行	25.1%
合计	654,763 行	100%

改造周期：项目启动、三端重构、上线恢复至实验推全，共 2 个月

本页说明改造范围与投入规模，下一页看业务迭代的效率与质量

业务迭代效果：效率与线上质量

需求吞吐率与线上 Bug 数量，均对比去年同周期

+320%

效率：需求吞吐率同比提升

-80%

质量：线上 Bug 数量同比减少

维度	实际结果	同比说明
迭代效率	需求吞吐率提升 320%	达到去年同周期的 4.2 倍
线上质量	线上 Bug 数量减少 80%	降至去年同周期的 20%

统计口径：两项指标均同比去年同周期（YoY）

结果反映整体工程实践，架构、编排、验证流程与人工决策共同影响交付表现

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

架构、生码、验证共同保障质量

逻辑可单测，行为可观察，交付结论有证据

